Chiron

Foundation Hardening for Methodology Contracts

Progress Report 11 • 15 Feb 2026 — 28 Feb 2026

Master’s Research Progress Update

Fahad Alothman

Progress Report 11

Title: Foundation Hardening for Methodology Contracts
Reporting Window: 15 Feb 2026 — 28 Feb 2026
Submission Context: Master’s Research Progress Update

Abstract

This reporting window marks the transition from architectural clarification into architectural enforcement. If the previous period made Chiron easier to describe, this one made it harder to violate. The project moved from explaining the layered model to hardening it across contracts, persistence, lifecycle semantics, and runtime ownership boundaries.

The main achievement of this period was not merely higher delivery volume. It was a stronger coupling between methodology authority and runtime behavior. L1 remained the methodology-wide authority layer, L2 became more operational as the work-unit contract layer, and L3 was formalized further as workflow grammar without overstating full end-to-end completion. By the end of the window, Chiron’s architecture was no longer just a good theory. It had started to become a more enforceable system.

From Checkpoint to Hardening

PR10 solved a descriptive problem: the project needed a coherent way to explain itself. PR11 addressed the harder problem that follows immediately after that: once the model is clarified, how does the system stop drifting away from it?

That question shaped the entire period. Repository evidence in this window shows broad landing across contracts, persistence, transition logic, publish and pinning behavior, and selected UI adaptation. At the same time, advisory and exploration work remained active in parallel. This dual-track pattern matters. The project was not improvising blindly. It was shipping under architectural pressure while continuing to validate where the boundaries should harden first.

What Actually Hardened in This Period

The major movement in this window was the conversion of architecture into ownership boundaries.

L1 stayed responsible for methodology topology and version-owned registries. That includes the methodology-wide definitions that should remain stable and reusable across projects. L2 became much more clearly the work-unit contract layer, where work-unit facts, lifecycle states, transitions, gates, workflow bindings, and artifact-slot policy belong. L3 was pushed further toward an explicit workflow grammar with typed step taxonomy, edges, and context-fact modeling, even while some of its runtime realization remained staged for later work.

This mattered because it reduced cross-layer leakage. Work-unit policy no longer needed to be smuggled through workflow-only concepts. Methodology-wide meaning no longer needed to be recreated inside each work unit. Runtime behavior became easier to reason about because there was a stronger answer to the question, “where does this piece of authority belong?”

Facts, Dependencies, and Contract Authority

One of the most important design consequences of this period was the sharpening of fact semantics. Methodology facts, work-unit facts, and runtime fact instances became harder to confuse. The project increasingly treated methodology facts as methodology-wide or project-wide durable inputs, while work-unit facts remained the place where work-unit-specific meaning and references could live.

This also reinforced a deeper architectural decision: dependency semantics belong to methodology data rather than engine enums. A work-unit reference is not merely a pointer. It gains meaning through methodology-defined link or dependency types. That decision keeps runtime generic while preserving expressiveness in the authored method. It also makes dependency interpretation part of the methodology contract rather than hardcoded logic spread across execution surfaces.

This is one of the most important reasons PR11 matters. It continued the move away from a workflow-first worldview and toward a methodology-driven one, where contracts govern how work relates, how it progresses, and how that meaning survives at runtime.

Lifecycle, Gates, and Runtime Determinism

The hardening work in this period also strengthened lifecycle authority. States and transitions were treated less like descriptive labels and more like binding policy. Gates became more explicit

as condition-set based readiness and completion checks. Transition execution behavior, workflow attachment, and completion semantics moved closer to a model where runtime must justify progression rather than simply allow it.

That had a direct impact on determinism. Publish and pinning behavior became more important because project runtime now depended more clearly on a stable methodology version. This also increased the value of treating runtime records as auditable outcomes of policy, not merely ephemeral events inside a workflow engine.

Artifact Evidence Becomes More Serious

Another important shift in this window was the strengthening of artifact thinking. Artifact slots became easier to understand as contract-owned definitions rather than loose attachment points. The key distinction was not the optional rules payload, but the simpler and more durable contract between `single` and `fileset` outputs.

On the runtime side, artifact behavior increasingly pointed toward immutable snapshots, file-member rows, and lineage rather than a mutable artifact row. That gave Chiron a more serious evidence model. When artifact members carry commit/blob references and can later be checked against project state, the result is not just storage. It is runtime evidence that can support readiness checks, auditability, and downstream consumption by other workflows.

This was a meaningful design choice because it aligned artifacts with the same larger principle that governed facts and lifecycle policy: durable project state should be explicit, attributable, and interpretable, not hidden inside ad hoc workflow IO.

Harness-First Practicality Begins to Dominate

PR11 also clarified an important product decision, even if it was not yet the final statement of it. The project increasingly favored a harness-oriented execution model for individual local users. This was not a rejection of richer AI-runtime ideas in principle. It was a judgment about maturity and utility.

The practical reasoning was straightforward. Harness-first execution already offered immediate value for local usage and simpler integration paths, while deeper AI-SDK and AX-centric infrastructure remained only partially scaffolded. Building the core system around those incomplete pieces would have increased conceptual ambition while weakening delivery reliability. This period therefore strengthened the case for a more pragmatic baseline: get the durable

execution model right first, then expand the AI-runtime stack when the underlying contracts and runtime evidence model are ready to support it. In that sense, OpenCode already had a real narrative role here as the practical harness boundary under active concretization [2].

Why This Window Matters

PR11 matters because it moved the project across a threshold. After this period, Chiron could no longer be described honestly as only an evolving design concept. Its contract model was becoming harder to bypass, its ownership boundaries were clearer, and its runtime behavior was increasingly tied to methodology authority instead of convenience logic.

That is the deeper contribution of this window. It made the project more structurally reliable. It also prepared the next period to do something more ambitious: summarize the current design of Chiron not as an aspirational architecture, but as a more concrete product direction with clear decisions about what to keep, what to defer, and what to reject.

Stack Direction at the End of the Window

The technical stack direction stayed stable even as implementation depth increased:

- **Desktop host:** Tauri [3]
- **Frontend:** React + TanStack Router + shadcn/ui
- **Backend/API:** TypeScript services with Hono + oRPC-aligned surfaces, while MCP transport direction becomes clearer through Hono-based MCP middleware, Hono itself, and the Model Context Protocol SDK [4]–[6]
- **Execution/control plane:** Effect-first with harness-oriented runtime direction, plus an increasingly concrete OpenCode SDK-backed harness boundary in `packages/agent-runtime` [1], [2]
- **Data layer:** Drizzle ORM with methodology-first persistence on a SQLite-first, libSQL/Turso-compatible baseline [7]
- **Git/workspace seam:** sandbox-oriented git truth direction, with simple-git still part of the intended git/workspace operations story even as the seam itself is being clarified [9]
- **Authentication:** Better Auth
- **Runtime/tooling:** Bun + Turborepo [8]

The signal here is not churn. It is hardening within an increasingly coherent architecture. Persistence choices followed the same logic. The project leaned into SQLite-first execution for the current horizon because it fit the local-first and desktop-friendly operating model, reduced

setup friction during high-velocity architecture work, and kept the team focused on contract correctness instead of distributed database overhead. At the same time, libSQL/Turso compatibility kept a future deployment path open without forcing an immediate second database track.

Conclusion

Progress Report 11 is the hardening phase in which Chiron's methodology-first design began to behave more like a system and less like a draft. L1 remained the authority layer for methodology topology, L2 became more enforceable as the work-unit contract layer, and L3 moved further toward explicit workflow grammar. Facts, dependencies, lifecycle policy, and artifact evidence all became more structurally grounded.

The result was stronger contract authority, stronger runtime determinism, and better readiness for the next step: explaining Chiron's current design as a deliberate product shape rather than an open-ended architecture experiment.

References

[1] Effect Authors, "Effect Documentation," 2026. [Online]. Available: https://effect.website/
[2] OpenCode, "OpenCode Documentation," 2026. [Online]. Available: https://opencode.ai/
[3] Tauri Programme, "Tauri," Documentation, 2026. [Online]. Available: https://tauri.app/
[4] Hono Authors, "Hono," Documentation, 2026. [Online]. Available: https://hono.dev/
[5] Hono, "@hono/mcp," npm, 2026. [Online]. Available: https://www.npmjs.com/package/@hono/mcp
[6] Model Context Protocol, "Introduction," 2026. [Online]. Available: https://modelcontextprotocol.io/docs/getting-started/intro
[7] Drizzle Team, "Drizzle ORM Documentation," 2026. [Online]. Available: https://orm.drizzle.team/
[8] Turborepo Authors, "Turborepo Documentation," 2026. [Online]. Available: https://turbo.build/repo/docs
[9] simple-git, "simple-git," GitHub repository, 2026. [Online]. Available: https://github.com/simple-git-js/simple-git/